

PROJECT REPORT

Inventory Management System

=====

1. ABSTRACT

A desktop application for maintaining product information, tracking stock quantities, searching products, and updating inventory. The system demonstrates how Python can be used to create a graphical desktop application connected to a relational database. It provides a practical example of database CRUD operations and event-driven GUI programming.

2. INTRODUCTION

Manual record management can be time-consuming and may lead to duplicate entries, calculation mistakes, and difficulty locating information. This project provides a computerized solution for the selected management task. The application has a simple interface so that users can enter, view, update, search, or process records without directly writing SQL commands.

3. PROBLEM STATEMENT

The traditional manual method of maintaining records requires physical registers or scattered files. Such methods can make data retrieval and maintenance difficult. The proposed system stores information in a structured SQLite database and provides a user-friendly graphical interface.

4. OBJECTIVES

- Store product information in a structured database.
- Add new products and stock quantities.
- Update stock after purchases or sales.
- Search products quickly.
- Delete obsolete product records.

5. SCOPE OF THE PROJECT

The project covers the basic operations required for a small-scale academic or departmental environment. It focuses on local data storage, simple record management, and easy interaction through a desktop GUI.

6. TECHNOLOGIES AND TOOLS

- Python 3
- Tkinter for graphical user interface
- SQLite3 for database management
- VS Code/IDLE/PyCharm or any Python IDE
- Operating systems: Windows, Linux, or macOS

7. SYSTEM REQUIREMENTS

Hardware:

- Basic computer/laptop
- Minimum 2 GB RAM recommended
- Small amount of free disk space

Software:

- Python 3.x
- Standard Python libraries used by the project

8. FUNCTIONAL MODULES

- Product Registration
- Product Search
- Stock Update
- Product Deletion
- Inventory Display

9. DATABASE DESIGN

Database file: inventory.db

The application creates the required SQLite table(s) automatically. Data is stored locally so that records remain available after the program is closed.

10. WORKING OF THE SYSTEM

1. Launch the Python program.
2. The application opens its Tkinter graphical interface.
3. The program connects to the SQLite database.
4. The user enters or selects information through the GUI.
5. Button actions execute database operations.
6. Updated information is displayed in the table on the screen.
7. The user can close the program and reopen it later; stored database records remain available.

11. INPUTS

The user provides the information required by the project, such as record names, identifiers, categories, quantities, dates, answers, or other relevant fields shown in the GUI.

12. OUTPUTS

The application displays stored records in a tabular format and performs the main calculations or status changes required by the project.

13. ADVANTAGES

- Simple graphical interface
- Local database storage
- Fast record retrieval
- Easy to run on a standard Python installation

- Reduces repetitive manual record handling
- Suitable for academic demonstration

14. LIMITATIONS

- Intended for small-scale use
- No advanced authentication system
- Local database only
- Limited validation compared with production software
- No cloud or multi-user synchronization

15. FUTURE ENHANCEMENTS

- Low-stock alerts
- Supplier management
- Sales and purchase invoices
- Barcode integration
- Dashboard and charts

16. TESTING

Test Case 1: Launch the application.

Expected Result: Main GUI opens successfully.

Test Case 2: Add a valid record.

Expected Result: Record is saved and appears in the table.

Test Case 3: Perform a main project operation.

Expected Result: Database and displayed data are updated correctly.

Test Case 4: Close and reopen the application.

Expected Result: Previously saved records remain available.

17. CONCLUSION

The Inventory Management System successfully demonstrates the use of Python, Tkinter, and SQLite to create a functional desktop management application. The project meets its basic objectives and provides a foundation that can be expanded with additional features in future versions.

18. REFERENCES / LEARNING SOURCES

- Python Standard Library documentation
- Tkinter documentation
- SQLite documentation
- Python SQLite3 module documentation

19. PROJECT FILES

- inventory_system.py — source code
- README.txt — setup and usage guide
- PROJECT_REPORT.txt — academic report
- inventory.db — generated SQLite database

20. STUDENT DETAILS

Name: Druvithareddy

Roll Number: 26MIM10178

Course/Class: integrated MTech ai

College: vit Bhopal university

Academic Year: 2026_2027